

# Rapport — Labo 03 : REST APIs, GraphQL

---

LOG430 – Architecture logicielle — ÉTS Auteur : Ralph Christian Gabriel

---

## Introduction

---

Ce laboratoire transforme l'application monolithique de gestion de magasin en une **API REST/GraphQL** avec Flask, MySQL et Redis. Deux domaines coexistent : `orders` (commandes) et `stocks` (stocks). Ce rapport documente l'implémentation des activités et répond aux six questions.

L'ensemble du système est conteneurisé (Docker Compose) et validé par un test d'intégration automatisé exécuté en CI/CD.

---

## Activité 1 — Diagramme ER et table `stocks`

---

La table `stocks` a été créée (MySQL Workbench, puis appliquée via *Forward Engineering*). Elle est aussi déclarée dans `db-init/init.sql` pour l'initialisation automatique du conteneur :

```
CREATE TABLE stocks (  
  product_id INT PRIMARY KEY,  
  quantity INT NOT NULL DEFAULT 0,  
  FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE RESTRICT  
);
```

**Relation :** `stocks.product_id` → `products.id` (1–1). La clé étrangère garantit qu'on ne peut ajouter du stock que pour un produit existant ; `ON DELETE RESTRICT` empêche la suppression d'un produit encore référencé.

Livrable : fichier `.mwb` joint (modèle ER complet : `users`, `products`, `orders`, `order_items`, `stocks`).

---

## Activité 2 — Test du processus de stock

---

Le test `test_stock_flow()` (`src/tests/test_store_manager.py`) implémente les 6 étapes :

| # | Action                        | Endpoint           | Vérification           |
|---|-------------------------------|--------------------|------------------------|
| 1 | Créer un article              | POST /products     | 201, product_id > 0    |
| 2 | Ajouter 5 unités              | POST /stocks       | 201                    |
| 3 | Vérifier le stock             | GET /stocks/:id    | quantity == 5          |
| 4 | Commander 2 unités            | POST /orders       | 201, order_id > 0      |
| 5 | Vérifier le stock             | GET /stocks/:id    | quantity == 3          |
| 6 | Supprimer la commande (extra) | DELETE /orders/:id | 200, stock remonte à 5 |

### Résultat d'exécution (dans le conteneur) :

```
tests/test_store_manager.py::test_health PASSED      [ 50%]
tests/test_store_manager.py::test_stock_flow PASSED [100%]
===== 2 passed in 2.54s =====
```

## 💡 Question 1 — Méthodes HTTP sûres / idempotentes (RFC 7231 §4.2.1–4.2.2)

- **Sûre (safe, §4.2.1)** : la méthode ne modifie pas l'état du serveur (lecture seule).
- **Idempotente (§4.2.2)** : exécuter la requête N fois produit le même état qu'une seule fois.

| Méthode | Endpoint(s) de l'activité 2                  | Sûre ? | Idempotente ? |
|---------|----------------------------------------------|--------|---------------|
| GET     | GET /stocks/:id                              | ✅ Oui  | ✅ Oui         |
| POST    | POST /products , POST /stocks , POST /orders | ❌ Non  | ❌ Non         |
| DELETE  | DELETE /orders/:id                           | ❌ Non  | ✅ Oui         |

**Justification** : - GET est **sûre et idempotente** : elle ne fait que lire le stock. - POST n'est **ni sûre ni idempotente** : chaque appel crée une nouvelle ressource (deux POST /orders identiques créent deux commandes et décrémentent le stock deux fois). - DELETE est **non sûre** (elle modifie l'état) mais **idempotente** : supprimer la commande #5 une fois ou plusieurs fois aboutit au même état final (la commande n'existe plus) ; les appels suivants renvoient 404 sans changer l'état.

Cas particulier de POST /stocks : l'implémentation fait un UPDATE ... SET quantity = :qty (affectation absolue), donc en pratique elle est idempotente. Mais **selon la sémantique RFC de POST**, la méthode reste classée non idempotente (le serveur n'en garantit pas l'idempotence).

## Activité 3 — Rapport de stock avec JOIN SQLAlchemy

`get_stock_for_all_products()` (`src/stocks/queries/read_stock.py`) joint `Stock` et `Product` pour ajouter `name`, `sku`, `price`.

### 💡 Question 2 — Utilisation de `join`

J'utilise la forme « **Joins to a Target with an ON Clause** » de SQLAlchemy : `query.join(cible, condition_ON)`. La cible est `Product`, et la condition `ON` est `Stock.product_id == Product.id` :

```
def get_stock_for_all_products():
    """Get stock quantity for all products (avec un JOIN vers Product)"""
    session = get_sqlalchemy_session()
    results = session.query(
        Stock.product_id,
        Stock.quantity,
        Product.name,
        Product.sku,
        Product.price,
    ).join(Product, Stock.product_id == Product.id).all()
    stock_data = []
    for row in results:
        stock_data.append({
            'Article': row.name,
            'Numero SKU': row.sku,
            'Prix unitaire': float(row.price),
            'Unites en stock': int(row.quantity),
        })
    return stock_data
```

Comme il n'existe pas de relation ORM ( `relationship` ) déclarée entre `Stock` et `Product`, on ne peut pas utiliser la forme *Simple Relationship Join* ( `join(Product)` implicite). On **précise donc explicitement la clause ON** via le second argument. SQLAlchemy génère :

```
SELECT stocks.product_id, stocks.quantity, products.name, products.sku, products.price
FROM stocks JOIN products ON stocks.product_id = products.id;
```

**Résultat ( GET /stocks/reports/overview-stocks ) :**

```
[
  { "Article": "Laptop ABC",          "Numero SKU": "LP12567", "Prix unitaire": 1999.99
```

```
{ "Article": "Keyboard DEF",      "Numero SKU": "KB67890", "Prix unitaire": 59.5,
  { "Article": "Gadget XYZ",      "Numero SKU": "GG12345", "Prix unitaire": 5.75,
    { "Article": "27-inch Screen WYZ", "Numero SKU": "SC27289", "Prix unitaire": 299.75,
      ]
}
```

## Activité 4 — Endpoint GraphQL

L'endpoint `POST /stocks/graphql-query` permet au client de demander exactement les champs voulus.

### 💡 Question 3 — Résultat initial

Avec le résolveur d'origine ( `resolve_product` ne lisant que `quantity` dans Redis, et le type GraphQL n'exposant que `id`, `name`, `quantity` ), la requête suggérée :

```
{ product(id: "1") { id quantity } }
```

retournait uniquement `id` et `quantity`. En demandant `name`, on obtenait la valeur **placeholder** `"Product 1"` (codée en dur), et les champs `sku / price` **n'existaient pas** dans le schéma (erreur *Cannot query field*). Les données réelles de l'article n'étaient donc pas disponibles via GraphQL.

## Activité 5 — Enrichissement de l'endpoint GraphQL

Trois changements : (a) ajout de `sku / price` au type GraphQL `Product`, (b) lecture de `name / sku / price` dans `resolve_product`, (c) écriture de ces champs dans Redis via `update_stock_redis` et `set_stock_for_product`.

### 💡 Question 4 — Lignes modifiées dans `update_stock_redis`

J'ai ouvert une session SQLAlchemy pour récupérer l'article et **construire un mapping** contenant `name`, `sku`, `price` en plus de `quantity`, puis je l'écris dans Redis via `hset(..., mapping=...)` :

```
# Ajout d'information sur l'article (name, sku, price) dans Redis
mapping = {"quantity": new_quantity}
product = session.query(Product).filter(Product.id == product_id).first()
if product:
    mapping["name"] = product.name
    mapping["sku"] = product.sku
```

```
mapping["price"] = float(product.price)

pipeline.hset(f"stock:{product_id}", mapping=mapping)
```

Lignes clés ajoutées par rapport à l'original (qui ne faisait que `pipeline.hset(f"stock:{product_id}", "quantity", new_quantity)`): 1. `import de Product` (modèle) en tête de fichier ; 2. ouverture d'une `session = get_sqlalchemy_session()` (fermée dans un `finally`) ; 3. requête `session.query(Product).filter(Product.id == product_id).first()` ; 4. construction du `mapping {quantity, name, sku, price}` ; 5. `pipeline.hset(..., mapping=mapping)` au lieu d'un champ unique.

La même logique d'enrichissement a été ajoutée à `set_stock_for_product` (chemin emprunté par `POST /stocks`).

## 💡 Question 5 — Résultat après améliorations

Requête :

```
{ product(id: "1") { id name sku price quantity } }
```

Réponse réelle de `POST /stocks/graphql-query` :

```
{
  "data": {
    "product": {
      "id": 1,
      "name": "Laptop ABC",
      "sku": "LP12567",
      "price": 1999.99,
      "quantity": 1000
    }
  },
  "errors": null
}
```

Le client peut désormais interroger `name`, `sku`, `price` et `quantity` via GraphQL, en choisissant exactement les champs dont il a besoin.

---

## Activité 6 — Communication inter-conteneurs

---

Le script fournisseur `scripts/supplier_app.py` interroge l'endpoint GraphQL depuis un conteneur séparé. Son `TEST_PAYLOAD` a été étendu pour inclure `sku` et `price` :

```
TEST_PAYLOAD = '{"query":"'{"\n  product(id: \\\"1\\\") {\n    id\n    name\n
```

**Execution du conteneur fournisseur** (`docker compose -f scripts/docker-compose.yml up --build`). Le conteneur, sur le réseau `labo03-network`, joint l'API par son **nom de service** `store_manager` :

```
INFO - Calling http://store_manager:5000/stocks/graphql-query (attempt 1/3)
INFO - Response: 200 - OK
INFO - Response body: {"data":{"product":{"id":1,"name":"Laptop ABC","price":1999.99,"
```

Le client fournisseur reçoit bien `name`, `sku` et `price`. `ENDPOINT_URL` a été rendu configurable (défaut `localhost`, surcharge vers `http://store_manager:5000/...` dans `scripts/docker-compose.yml`) afin que le conteneur joigne l'API via le DNS du réseau Docker.

## 💡 Question 6 — Mécanisme de communication

Les deux fichiers (`docker-compose.yml` à la racine et `scripts/docker-compose.yml`) déclarent **le même réseau externe** `labo03-network` :

```
networks:
  labo03-network:
    driver: bridge
    external: true
```

**Point commun** : les deux stacks rejoignent ce **réseau bridge défini par l'utilisateur**. Sur un tel réseau, Docker fournit un **DNS interne** qui résout chaque conteneur par son **nom de service**. Le conteneur `supplier_app` peut donc joindre l'API via le hostname `store_manager` (ou `log430-a25-labo3-store_manager` selon le nom), sans connaître son IP :

```
http://store_manager:5000/stocks/graphql-query
```

C'est ce qui permet aux conteneurs de communiquer entre eux comme s'ils étaient sur un même réseau local, tout en restant isolés du reste du système. (Vérifié : `getent hosts mysql` depuis `store_manager` → `172.21.0.3 mysql`.)

# Intégration CI/CD et conteneurisation

---

Le pipeline `.github/workflows/ci.yml` automatise les tests à chaque `push / pull_request` :

1. **Services** : conteneurs `mysql:8.4.7` et `redis:7` avec *health-checks*.
2. **Setup** : Python 3.11 + `pip install -r requirements.txt`.
3. **Configuration** : génération du `.env` (correction du bug `DB_PASS` → `DB_PASSWORD`).
4. **Initialisation BD** : `mysql ... < db-init/init.sql`.
5. **Tests** : `python -m pytest tests/ -v` (remplace le `echo` placeholder du gabarit).

Conteneurisation locale : `docker network create labo03-network` puis `docker compose up -d` démarre les 3 services (`store_manager`, `mysql`, `redis`), tous *healthy*. Le test d'intégration s'exécute via `docker compose exec store_manager python -m pytest tests/ -v`.

---

## Conclusion

---

Toutes les activités sont implémentées et **validées par exécution réelle** : test d'intégration vert (2/2), rapport de stock avec JOIN, endpoint GraphQL enrichi (name/sku/price), communication inter-conteneurs par réseau bridge nommé, et pipeline CI/CD fonctionnel. L'API illustre les principes REST (interface uniforme, sans état, client-serveur) complétés par la flexibilité de GraphQL pour la sélection des champs.